

Payments + Wallet + Card Fact Pack

A) Database Schema (Supabase/Postgres)

1) Full SQL Schema

drivers table

```
CREATE TABLE IF NOT EXISTS public.drivers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  auth_user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  full_name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  phone TEXT NOT NULL,
  city TEXT NOT NULL,
  zip TEXT NOT NULL,
  zone_id UUID REFERENCES public.zones(id),
  status TEXT NOT NULL DEFAULT 'started' CHECK (status IN (
    'started', 'consents_ok', 'id_submitted', 'pending_check',
    'awaiting_contract', 'contract_signed', 'waitlisted_contract_signed',
    'eligible', 'active', 'suspended', 'rejected'
  )),
  ssn_last4 TEXT,
  contract_signed_at TIMESTAMP WITH TIME ZONE,
  docusign_envelope_id TEXT,
  activated_at TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_drivers_auth_user ON public.drivers(auth_user_id);
CREATE INDEX IF NOT EXISTS idx_drivers_email ON public.drivers(email);
CREATE INDEX IF NOT EXISTS idx_drivers_status ON public.drivers(status);
CREATE INDEX IF NOT EXISTS idx_drivers_zone ON public.drivers(zone_id);
```

driver_gas_money table (wallet balance)

```
CREATE TABLE IF NOT EXISTS driver_gas_money (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  driver_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  balance INTEGER NOT NULL DEFAULT 0, -- Balance in cents
  total_accumulated INTEGER NOT NULL DEFAULT 0, -- Total ever accumulated in cents
  total_transferred INTEGER NOT NULL DEFAULT 0, -- Total ever transferred in cents
  last_earned_at TIMESTAMPTZ, -- Last time mileage was earned
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(driver_id)
```

```
);

CREATE INDEX IF NOT EXISTS idx_driver_gas_money_driver_id ON
driver_gas_money(driver_id);
```

driver_earnings table

```
CREATE TABLE public.driver_earnings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  driver_id UUID REFERENCES public.users(id) ON DELETE CASCADE,
  order_id UUID REFERENCES public.orders(id),
  amount_cents INTEGER NOT NULL,
  tip_cents INTEGER DEFAULT 0,
  total_cents INTEGER NOT NULL,
  payout_cents INTEGER NOT NULL,
  earned_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);
```

driver_payouts table

```
CREATE TABLE public.driver_payouts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  driver_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  stripe_account_id TEXT NOT NULL,
  stripe_payout_id TEXT UNIQUE NOT NULL,
  amount_cents INTEGER NOT NULL CHECK (amount_cents > 0),
  currency TEXT NOT NULL DEFAULT 'usd',
  payout_type TEXT NOT NULL CHECK (payout_type IN ('instant', 'standard')),
  status TEXT NOT NULL CHECK (status IN ('pending', 'in_transit', 'paid',
'failed', 'canceled')),
  arrival_date TIMESTAMP WITH TIME ZONE,
  failure_code TEXT,
  failure_message TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

CREATE INDEX IF NOT EXISTS driver_payouts_driver_id_idx ON
public.driver_payouts(driver_id);
CREATE INDEX IF NOT EXISTS driver_payouts_stripe_payout_id_idx ON
public.driver_payouts(stripe_payout_id);
CREATE INDEX IF NOT EXISTS driver_payouts_status_idx ON
public.driver_payouts(status);
CREATE INDEX IF NOT EXISTS driver_payouts_created_at_idx ON
public.driver_payouts(created_at DESC);
```

gas_money_transactions table (ledger)

```

CREATE TABLE IF NOT EXISTS gas_money_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  driver_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  order_id UUID REFERENCES orders(id) ON DELETE SET NULL,
  amount_cents INTEGER NOT NULL,
  transaction_type TEXT NOT NULL CHECK (transaction_type IN ('accumulate',
'transfer', 'adjustment', 'earned')),
  destination TEXT, -- 'feeder_card', 'bank', etc.
  description TEXT,
  status TEXT NOT NULL DEFAULT 'completed' CHECK (status IN ('pending',
'completed', 'failed')),
  notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_gas_money_transactions_driver_id ON
gas_money_transactions(driver_id);
CREATE INDEX IF NOT EXISTS idx_gas_money_transactions_created_at ON
gas_money_transactions(created_at DESC);

```

stripe_accounts table (stores Stripe account IDs)

```

CREATE TABLE IF NOT EXISTS public.stripe_accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_type TEXT NOT NULL CHECK (owner_type IN ('restaurant', 'driver')),
  owner_id UUID NOT NULL,
  stripe_account_id TEXT UNIQUE NOT NULL,
  details_submitted BOOLEAN DEFAULT FALSE,
  payouts_enabled BOOLEAN DEFAULT FALSE,
  charges_enabled BOOLEAN DEFAULT FALSE,
  requirements JSONB DEFAULT '{}',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  UNIQUE(owner_type, owner_id)
);

CREATE INDEX IF NOT EXISTS stripe_accounts_owner_idx ON
public.stripe_accounts(owner_type, owner_id);
CREATE INDEX IF NOT EXISTS stripe_accounts_stripe_id_idx ON
public.stripe_accounts(stripe_account_id);
CREATE INDEX IF NOT EXISTS stripe_accounts_payouts_enabled_idx ON
public.stripe_accounts(payouts_enabled) WHERE payouts_enabled = TRUE;

```

ledger_entries table

```

CREATE TABLE IF NOT EXISTS public.ledger_entries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id UUID REFERENCES public.orders(id) ON DELETE CASCADE,
  entry_type TEXT NOT NULL CHECK (entry_type IN (
    'customer_charge', 'platform_fee', 'restaurant_net', 'driver_pay',
    'tip', 'refund', 'dispute_debit', 'dispute_credit', 'adjustment'
  )),
  owner_type TEXT NOT NULL CHECK (owner_type IN ('platform', 'restaurant',
'driver')),
  owner_id UUID,
  amount_cents INTEGER NOT NULL,
  currency TEXT NOT NULL DEFAULT 'usd',
  stripe_object_id TEXT,
  memo TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  UNIQUE(order_id, entry_type, owner_type, owner_id)
);

CREATE INDEX IF NOT EXISTS ledger_entries_order_idx ON
public.ledger_entries(order_id);
CREATE INDEX IF NOT EXISTS ledger_entries_owner_idx ON
public.ledger_entries(owner_type, owner_id);
CREATE INDEX IF NOT EXISTS ledger_entries_type_idx ON
public.ledger_entries(entry_type);
CREATE INDEX IF NOT EXISTS ledger_entries_created_idx ON
public.ledger_entries(created_at DESC);

```

2) RLS Policies

driver_gas_money policies

```

CREATE POLICY "Drivers can view their own gas money"
ON driver_gas_money FOR SELECT
USING (auth.uid() = driver_id);

CREATE POLICY "Drivers can update their own gas money"
ON driver_gas_money FOR UPDATE
USING (auth.uid() = driver_id);

CREATE POLICY "System can insert gas money records"
ON driver_gas_money FOR INSERT
WITH CHECK (true);

```

gas_money_transactions policies

```

CREATE POLICY "Drivers can view their own gas money transactions"
ON gas_money_transactions FOR SELECT
USING (auth.uid() = driver_id);

```

```
CREATE POLICY "System can insert gas money transactions"
ON gas_money_transactions FOR INSERT
WITH CHECK (true);
```

driver_payouts policies

```
CREATE POLICY "drivers_can_view_own_payouts" ON public.driver_payouts
FOR SELECT
USING (auth.uid() = driver_id);

CREATE POLICY "service_role_can_manage_payouts" ON public.driver_payouts
FOR ALL
USING (auth.jwt() ->> 'role' = 'service_role');

CREATE POLICY "admin_full_access_payouts" ON public.driver_payouts
FOR ALL
USING (
  auth.jwt() ->> 'email' = 'tstroman.ceo@cravenusa.com'
  OR EXISTS (
    SELECT 1 FROM public.user_roles
    WHERE user_roles.user_id = auth.uid()
    AND user_roles.role IN ('admin', 'ceo', 'super_admin')
  )
);
```

stripe_accounts policies

```
CREATE POLICY "stripe_accounts_admin_access" ON public.stripe_accounts
FOR ALL
USING (
  auth.jwt() ->> 'email' = 'tstroman.ceo@cravenusa.com'
  OR EXISTS (
    SELECT 1 FROM public.user_roles
    WHERE user_roles.user_id = auth.uid()
    AND user_roles.role IN ('admin', 'ceo', 'super_admin')
  )
);
```

ledger_entries policies

```
CREATE POLICY "ledger_entries_admin_access" ON public.ledger_entries
FOR ALL
USING (
  auth.jwt() ->> 'email' = 'tstroman.ceo@cravenusa.com'
  OR EXISTS (
```

```

        SELECT 1 FROM public.user_roles
        WHERE user_roles.user_id = auth.uid()
        AND user_roles.role IN ('admin', 'ceo', 'super_admin')
    )
);

CREATE POLICY "ledger_entries_owner_read" ON public.ledger_entries
FOR SELECT
USING (
    (owner_type = 'restaurant' AND owner_id = auth.uid())
    OR (owner_type = 'driver' AND owner_id = auth.uid())
);

```

3) RPC Functions

accumulate_gas_money

```

CREATE OR REPLACE FUNCTION accumulate_gas_money(
    p_driver_id UUID,
    p_amount_cents INTEGER
)
RETURNS void AS $$
BEGIN
    INSERT INTO driver_gas_money (driver_id, balance, total_accumulated)
    VALUES (p_driver_id, p_amount_cents, p_amount_cents)
    ON CONFLICT (driver_id)
    DO UPDATE SET
        balance = driver_gas_money.balance + p_amount_cents,
        total_accumulated = driver_gas_money.total_accumulated + p_amount_cents,
        updated_at = NOW();

    INSERT INTO gas_money_transactions (driver_id, amount_cents, transaction_type,
status)
    VALUES (p_driver_id, p_amount_cents, 'accumulate', 'completed');
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

GRANT EXECUTE ON FUNCTION accumulate_gas_money TO authenticated;

```

accumulate_mileage_pay_on_delivery (trigger function)

```

CREATE OR REPLACE FUNCTION accumulate_mileage_pay_on_delivery()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.order_status = 'delivered' AND (OLD.order_status IS NULL OR
OLD.order_status != 'delivered') THEN
        IF NEW.driver_id IS NOT NULL AND NEW.mileage_pay_cents > 0 THEN
            INSERT INTO driver_gas_money (driver_id, balance, last_earned_at,

```

```

updated_at)
    VALUES (NEW.driver_id, NEW.mileage_pay_cents, NOW(), NOW())
    ON CONFLICT (driver_id)
    DO UPDATE SET
        balance = driver_gas_money.balance + NEW.mileage_pay_cents,
        last_earned_at = NOW(),
        updated_at = NOW();

    INSERT INTO gas_money_transactions (
        driver_id, order_id, amount_cents, transaction_type, description,
        created_at
    )
    VALUES (
        NEW.driver_id, NEW.id, NEW.mileage_pay_cents, 'earned', 'Mileage pay from
delivery', NOW()
    );
    END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

calculate_driver_daily_earnings

```

CREATE OR REPLACE FUNCTION public.calculate_driver_daily_earnings(
    target_driver_id UUID,
    target_date DATE
) RETURNS DECIMAL(10,2)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    total_earnings DECIMAL(10,2) := 0;
BEGIN
    SELECT COALESCE(SUM(amount), 0)
    INTO total_earnings
    FROM public.driver_earnings
    WHERE driver_id = target_driver_id
        AND DATE(created_at) = target_date;
    RETURN total_earnings;
END;
$$;

```

B) Current Payout Code

1) Full Edge Function Contents

supabase/functions/transfer-earnings/index.ts

```
import { serve } from 'https://deno.land/std@0.168.0/http/server.ts';
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2';

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
};

serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response(null, { headers: corsHeaders });
  }

  try {
    const supabaseClient = createClient(
      Deno.env.get('SUPABASE_URL') ?? '',
      Deno.env.get('SUPABASE_ANON_KEY') ?? '',
      {
        global: {
          headers: { Authorization: req.headers.get('Authorization')! },
        },
      }
    );

    const {
      data: { user },
      error: userError,
    } = await supabaseClient.auth.getUser();

    if (userError || !user) {
      throw new Error('Unauthorized');
    }

    const { driver_id, amount_cents } = await req.json();

    if (!driver_id || !amount_cents || amount_cents <= 0) {
      throw new Error('Invalid transfer amount');
    }

    if (user.id !== driver_id) {
      throw new Error('Unauthorized: Cannot transfer earnings for another driver');
    }

    const { data: earningsData, error: earningsError } = await supabaseClient
      .from('driver_earnings')
      .select('total_cents')
      .eq('driver_id', driver_id);

    if (earningsError) {
      throw new Error(`Failed to fetch earnings: ${earningsError.message}`);
    }
  }
});
```



```
}

const totalEarnings = earningsData?.reduce((sum, e) => sum + e.total_cents, 0)
|| 0;

const { data: payoutsData, error: payoutsError } = await supabaseClient
  .from('driver_payouts')
  .select('amount_cents, status')
  .eq('driver_id', driver_id)
  .in('status', ['pending', 'in_transit', 'paid']);

if (payoutsError) {
  throw new Error(`Failed to fetch payouts: ${payoutsError.message}`);
}

const totalPaidOut = payoutsData?.reduce((sum, p) => sum + p.amount_cents, 0)
|| 0;
const availableBalance = totalEarnings - totalPaidOut;

if (availableBalance < amount_cents) {
  throw new Error(`Insufficient available balance. Available:
${(availableBalance / 100).toFixed(2)}`);
}

const { data: payoutRecord, error: payoutError } = await supabaseClient
  .from('driver_payouts')
  .insert({
    driver_id,
    stripe_account_id: 'feeder_card_placeholder',
    stripe_payout_id: `po_feeder_${Date.now()}_${driver_id.slice(0, 8)}`,
    amount_cents,
    currency: 'usd',
    payout_type: 'instant',
    status: 'paid',
    arrival_date: new Date().toISOString(),
  })
  .select()
  .single();

if (payoutError) {
  throw new Error(`Failed to create payout record: ${payoutError.message}`);
}

const newAvailableBalance = availableBalance - amount_cents;

return new Response(
  JSON.stringify({
    success: true,
    message: 'Earnings transferred successfully',
    amount_cents,
    new_available_balance: newAvailableBalance,
    payout_id: payoutRecord.id,
  }),
  {

```

```

        headers: { ...corsHeaders, 'Content-Type': 'application/json' },
        status: 200,
      }
    );
  } catch (error) {
    console.error('Error transferring earnings:', error);
    return new Response(
      JSON.stringify({
        error: error.message || 'Failed to transfer earnings',
      }),
      {
        headers: { ...corsHeaders, 'Content-Type': 'application/json' },
        status: 400,
      }
    );
  }
});

```

supabase/functions/transfer-gas-money/index.ts

```

import { serve } from 'https://deno.land/std@0.168.0/http/server.ts';
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2';

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
};

serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response(null, { headers: corsHeaders });
  }

  try {
    const supabaseClient = createClient(
      Deno.env.get('SUPABASE_URL') ?? '',
      Deno.env.get('SUPABASE_ANON_KEY') ?? '',
      {
        global: {
          headers: { Authorization: req.headers.get('Authorization')! },
        },
      }
    );

    const {
      data: { user },
      error: userError,
    } = await supabaseClient.auth.getUser();

    if (userError || !user) {

```

```
    throw new Error('Unauthorized');
  }

  const { driver_id, amount_cents } = await req.json();

  if (!driver_id || !amount_cents || amount_cents <= 0) {
    throw new Error('Invalid transfer amount');
  }

  if (user.id !== driver_id) {
    throw new Error('Unauthorized: Cannot transfer gas money for another driver');
  }

  const { data: gasMoneyData, error: gasMoneyError } = await supabaseClient
    .from('driver_gas_money')
    .select('balance')
    .eq('driver_id', driver_id)
    .single();

  if (gasMoneyError || !gasMoneyData) {
    throw new Error('Gas money account not found');
  }

  if (gasMoneyData.balance < amount_cents) {
    throw new Error('Insufficient gas money balance');
  }

  const { error: updateError } = await supabaseClient
    .from('driver_gas_money')
    .update({
      balance: gasMoneyData.balance - amount_cents,
      total_transferred: supabaseClient.rpc('increment', {
        row_id: driver_id,
        amount: amount_cents,
      }),
    })
    .eq('driver_id', driver_id);

  if (updateError) {
    throw new Error(`Failed to update gas money balance: ${updateError.message}`);
  }

  const { error: transactionError } = await supabaseClient
    .from('gas_money_transactions')
    .insert({
      driver_id,
      amount_cents,
      transaction_type: 'transfer',
      destination: 'feeder_card',
      status: 'completed',
      notes: 'Transferred to Feeder Card',
    });
```

```

    if (transactionError) {
      console.error('Failed to record transaction:', transactionError);
    }

    return new Response(
      JSON.stringify({
        success: true,
        message: 'Gas money transferred successfully',
        amount_cents,
        new_balance: gasMoneyData.balance - amount_cents,
      }),
      {
        headers: { ...corsHeaders, 'Content-Type': 'application/json' },
        status: 200,
      }
    );
  } catch (error) {
    console.error('Error transferring gas money:', error);
    return new Response(
      JSON.stringify({
        error: error.message || 'Failed to transfer gas money',
      }),
      {
        headers: { ...corsHeaders, 'Content-Type': 'application/json' },
        status: 400,
      }
    );
  }
});

```

2) Authentication & driver_id Fetching

Both functions authenticate identically:

- Extract `Authorization` header from request
- Create Supabase client with `SUPABASE_ANON_KEY` and auth header
- Call `supabaseClient.auth.getUser()` to get authenticated user
- Extract `driver_id` from request JSON body
- Validate `user.id === driver_id` to ensure user can only transfer their own funds

3) driver_stripe_account_id Storage & Reading

Storage:

- Stored in `stripe_accounts` table
- Column: `stripe_account_id` (TEXT)
- Lookup: `owner_type = 'driver' AND owner_id = driver_id`

Reading:

```
const { data: driverAccount } = await supabase
  .from('stripe_accounts')
  .select('stripe_account_id')
  .eq('owner_type', 'driver')
  .eq('owner_id', driver_id)
  .single();
```

Current Status:

- `transfer-earnings` uses placeholder: `'feeder_card_placeholder'`
- `transfer-gas-money` does not fetch or use `stripe_account_id` at all
- Both functions have TODO comments for Stripe integration

C) Stripe Integration Status

1) Environment Variables

Names:

- `STRIPE_SECRET_KEY` - Referenced in:
 - `supabase/functions/stripe-webhook/index.ts`
 - `supabase/functions/create-connected-account/index.ts`
 - `supabase/functions/create-stripe-connect-link/index.ts`
 - `supabase/functions/get-driver-balance/index.ts`
 - `supabase/functions/_shared/stripe.ts`
- `STRIPE_PUBLISHABLE_KEY` - Referenced in:
 - `supabase/functions/_shared/stripe.ts`
- `STRIPE_WEBHOOK_SECRET` - Referenced in:
 - `supabase/functions/stripe-webhook/index.ts`

2) Stripe Helper Libs/Wrappers

File: `supabase/functions/_shared/stripe.ts`

- Exports: `getStripeConfig()`, `getStripeClient()`, `createStripePaymentMethod()`, `attachPaymentMethodToCustomer()`, `getOrCreateCustomer()`, `createPaymentIntent()`, `confirmPaymentIntent()`, `createStripeTransfer()`, `createPayoutToConnectedAccount()`, `retrievePaymentMethod()`, `listCustomerPaymentMethods()`, `deletePaymentMethod()`
- Uses: `STRIPE_SECRET_KEY`, `STRIPE_PUBLISHABLE_KEY`
- API Version: `'2023-10-16'`

3) Stripe Connect Status

YES - Stripe Connect is live

Code Location: `supabase/functions/create-connected-account/index.ts`

Creates Connected Accounts:

```
const account = await stripe.accounts.create({
  type: 'express',
  country: 'US',
  email: email,
  capabilities: {
    transfers: { requested: true }, // ONLY transfers (no card_payments)
  },
  business_type: owner_type === 'restaurant' ? 'company' : 'individual',
  metadata: { owner_type, owner_id, platform: 'cravenusa' },
});
```

Stores stripe_account_id:

```
await supabase
  .from('stripe_accounts')
  .insert({
    owner_type, // 'restaurant' or 'driver'
    owner_id,
    stripe_account_id: account.id,
    details_submitted: account.details_submitted || false,
    payouts_enabled: account.payouts_enabled || false,
    charges_enabled: account.charges_enabled || false,
    requirements: account.requirements || {},
  });
```

Table: `stripe_accounts` (see schema in A.1)

D) Card Plan Readiness (Issuing)

Search Results

"issuing": No matches found

"cardholder": No matches found

"digital wallet": No matches found

"apple pay": No matches found

"google pay": No matches found

"authorization":

- Found in: `supabase/functions/stripe-webhook/index.ts` (line 22: `stripe-signature` header)
- Context: Webhook signature verification only

"webhook":

- File: `supabase/functions/stripe-webhook/index.ts`
- Endpoint: `/functions/v1/stripe-webhook`

- Verification: Uses `stripe.webhooks.constructEvent(body, signature, webhookSecret)`

"stripe-signature":

- File: `supabase/functions/stripe-webhook/index.ts` (line 22)
- Usage: `req.headers.get('stripe-signature')` for webhook verification

Webhook Handler

File: `supabase/functions/stripe-webhook/index.ts`

Endpoint Route: `/functions/v1/stripe-webhook`

Verification Logic:

```
const signature = req.headers.get('stripe-signature');
const webhookSecret = Deno.env.get('STRIPE_WEBHOOK_SECRET');

if (!signature || !webhookSecret) {
  return new Response('Missing signature', { status: 400 });
}

const body = await req.text();
const event = stripe.webhooks.constructEvent(body, signature, webhookSecret);
```

Deduplication:

- Uses `stripe_events` table with `event_id` as PRIMARY KEY
- INSERT-first pattern: if `23505` (duplicate key), returns early
- Updates status to 'processed' after successful handling

Handled Events:

- `account.updated` - Updates `stripe_accounts` table
- `payment_intent.succeeded` - Creates transfers, writes ledger
- `payment_intent.payment_failed` - Marks order as failed
- `charge.refunded` - Handles refunds
- `payout.paid`, `payout.failed`, `payout.canceled` - Updates `driver_payouts`
- `checkout.session.completed` - CraveMore subscriptions
- `customer.subscription.*` - Subscription lifecycle

E) Wallet Behavior Today

Balance Changes

1. When earnings are credited:

- Order delivered → `driver_earnings` record created with `total_cents`
- Mileage pay → `driver_gas_money.balance` incremented via trigger `accumulate_mileage_pay_on_delivery()`

- Gas money transaction logged in `gas_money_transactions` with type 'earned'

2. When payouts are requested:

- `transfer-earnings`: Creates `driver_payouts` record, status='paid' (placeholder)
- `transfer-gas-money`: Decrements `driver_gas_money.balance`, increments `total_transferred`
- Both update database but do NOT execute Stripe transfers (TODOs present)

3. Available vs Pending:

- **Available**: Calculated as `SUM(driver_earnings.total_cents) - SUM(driver_payouts.amount_cents WHERE status IN ['pending', 'in_transit', 'paid'])`
- **Pending**: `driver_payouts` records with status='pending' or 'in_transit'
- **Paid**: `driver_payouts` records with status='paid'

4. Holds/Reserved:

- No explicit holds/reserved balance mechanism
- Available balance is calculated on-the-fly, not stored

5. Gas Money Balance:

- Stored in `driver_gas_money.balance` (INTEGER cents)
- Separate from earnings balance
- Can be transferred independently

6. Earnings Balance:

- Not stored directly
- Calculated from `driver_earnings` minus `driver_payouts`
- No single "wallet" table

7. Payout Status States:

- `pending` - Created but not processed
- `in_transit` - Being processed
- `paid` - Successfully paid
- `failed` - Failed
- `canceled` - Canceled

8. Transfer Flow:

- User requests transfer → Edge function validates balance → Creates payout record → Updates local state → Returns success
- **Missing**: Actual Stripe API call to move funds

9. Balance Updates:

- Gas money: Updated via trigger on `orders` table when status='delivered'
- Earnings: Inserted into `driver_earnings` when order completes

- Payouts: Inserted into `driver_payouts` when transfer requested

10. No Real-Time Sync:

- Feeder Card balance fetched from Stripe via `get-driver-balance` function
- Uses `stripe.balance.retrieve({ stripeAccount: stripeAccountId })`
- Not automatically synced with internal wallet balances

Balance Update Locations

File: `supabase/migrations/20260201140000_accumulate_mileage_on_delivery.sql`

- Function: `accumulate_mileage_pay_on_delivery()` (trigger)
- Updates: `driver_gas_money.balance`

File: `supabase/functions/transfer-gas-money/index.ts`

- Function: `handleTransferGasMoney` (via edge function)
- Updates: `driver_gas_money.balance`, `gas_money_transactions`

File: `supabase/functions/transfer-earnings/index.ts`

- Function: `handleTransferEarnings` (via edge function)
- Inserts: `driver_payouts` record

File: `supabase/functions/finalize-delivery/index.ts` (referenced in search)

- Function: Creates `driver_earnings` records
- Inserts: `driver_earnings` table

File: `supabase/functions/stripe-webhook/index.ts`

- Function: `handlePaymentSucceeded()`
- Updates: Creates transfers, writes `ledger_entries`

F) Constraints & Requirements

1) Is `available_cents` stored as integer cents?

Answer: NO

Field Type:

- `driver_gas_money.balance`: INTEGER (cents) ☒
- `driver_earnings.total_cents`: INTEGER (cents) ☒
- `driver_payouts.amount_cents`: INTEGER (cents) ☒
- **BUT:** Available balance is NOT stored - it's calculated on-the-fly

Calculation:

```
const totalEarnings = SUM(driver_earnings.total_cents);
const totalPaidOut = SUM(driver_payouts.amount_cents WHERE status IN ['pending',
```

```
'in_transit', 'paid']]);  
const availableBalance = totalEarnings - totalPaidOut; // Calculated, not stored
```

2) Do we already have a unique "feeder_id" uuid for every driver?

Answer: YES

Field: `drivers.auth_user_id` (UUID, references `auth.users(id)`)

- One-to-one relationship: Each driver has one `auth_user_id`
- Used as `driver_id` in all wallet/payout tables
- Also stored as `driver_id` in `driver_earnings`, `driver_gas_money`, `driver_payouts`

Alternative: `drivers.id` (UUID) exists but `auth_user_id` is the primary identifier used in wallet tables

3) Are there any existing "transaction idempotency keys" used?

Answer: YES

Location: `supabase/functions/stripe-webhook/index.ts`

Usage:

```
// Line 245  
{ idempotencyKey: `order:${order_id}:transfer:restaurant` }  
  
// Line 268  
{ idempotencyKey: `order:${order_id}:transfer:driver` }
```

Pattern:

- Format: `order:{order_id}:transfer:{restaurant|driver}`
- Used when creating Stripe transfers
- Prevents duplicate transfers if webhook retries

Webhook Deduplication:

- Uses `stripe_events` table with `event_id` PRIMARY KEY
- INSERT-first pattern prevents duplicate processing
- Error code `23505` (duplicate key) = already processed

NOT Used:

- `transfer-earnings` function: No idempotency keys
- `transfer-gas-money` function: No idempotency keys
- Both functions could create duplicate payout records on retry